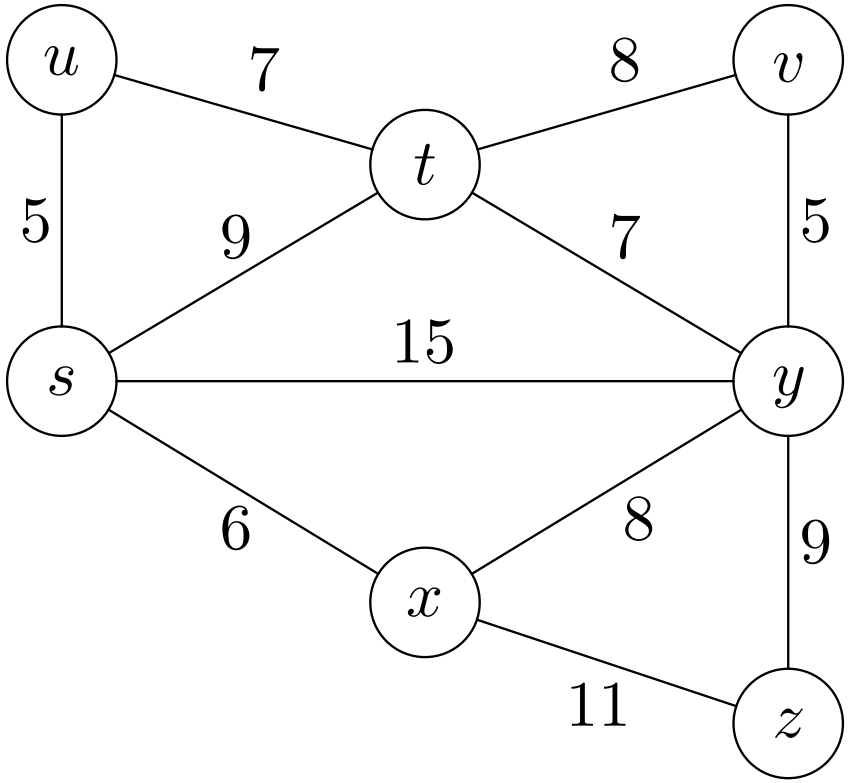
Minimum Spanning Tree: Kruskal's Algorithm (ADA 2023, CSE 222)

Diptapriyo Majumdar

March 20, 2022

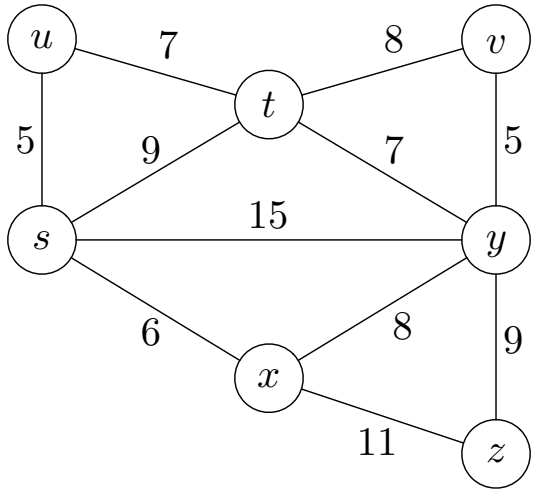
Spanning Tree

- Given a connected undirected graph $G = (V, E)$, spanning tree of G is $T = (V, F)$ such that $F \subseteq E(G)$ and T is a tree (i.e. connected and has no cycle).



Spanning Tree

- Given a connected undirected graph $G = (V, E)$, spanning tree of G is $T = (V, F)$ such that $F \subseteq E(G)$ and T is a tree (i.e. connected and has no cycle).
- Input: A connected, edge-weighted undirected graph $G = (V, E)$ and $\mathrm{w} : E(G) \to \mathbb{R}^+$.
- Goal: Compute a spanning tree with minimum total weight, i.e. compute a tree $T = (V, F)$ such that $\sum_{(u,v)\in F} \mathrm{w}[(u, v)]$ is minimized.



Kruskal's Algorithm

Input graph: n vertices and m edges.

Sort the edges in the nondecreasing order of weights;

Let $\{e_1, \ldots, e_m\}$ *be the nondecreasing order of weights*;

Initialize $F \leftarrow \emptyset; i \leftarrow 1$;

while F *does not form a spanning tree* **do**

  if *addition of* e_i *does not create cycle* **then**

    $F \leftarrow F \cup \{e_i\}$;

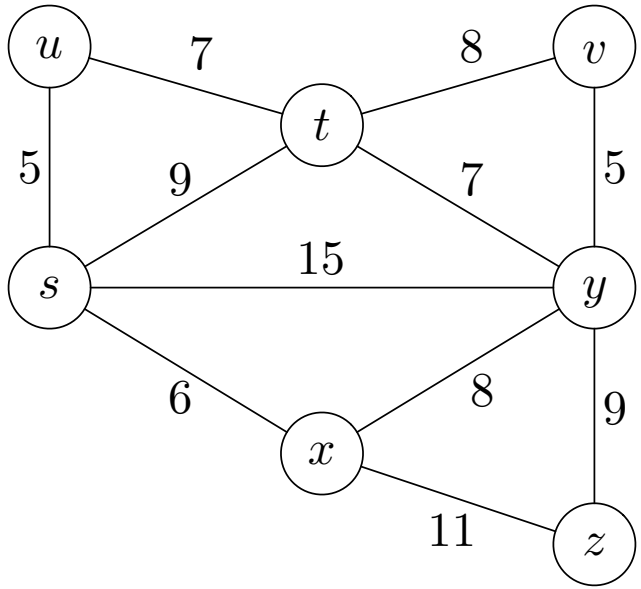
  end

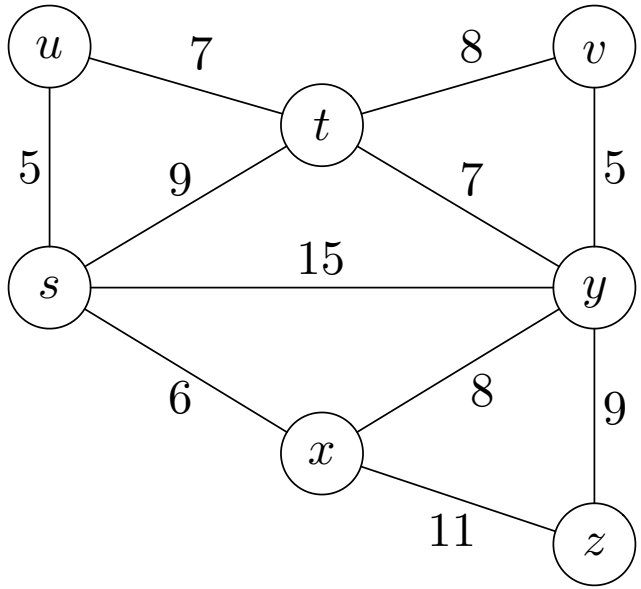
  $i \leftarrow i + 1$;

end

Return (V, F);

Running time: $\mathcal{O}(m \log m + n^2 m(n+m)^2)$.

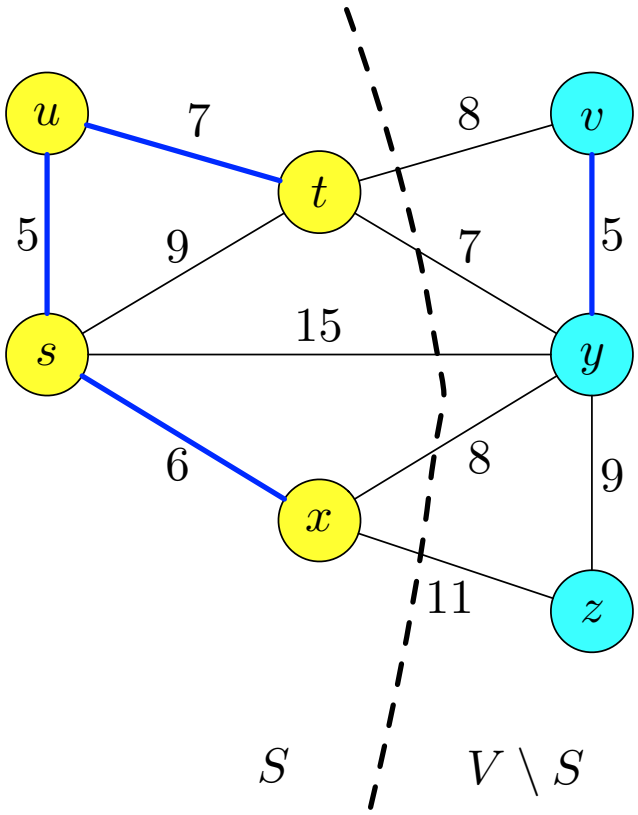

u
v
t
s
y
x
z
7
8
5
9
7
5
15
6
8
9
11

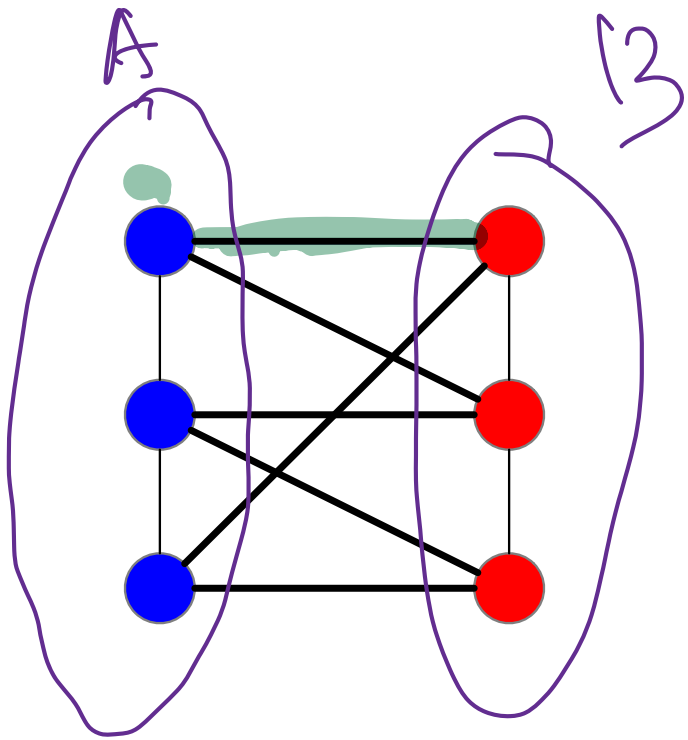


1. Invariant: at any intermediate step, $F \subseteq T$ for some minimum spanning tree T.
2. In other words, as long as F is not a tree, there is an edge uv addition of which does not create a cycle.

Proof Ideas

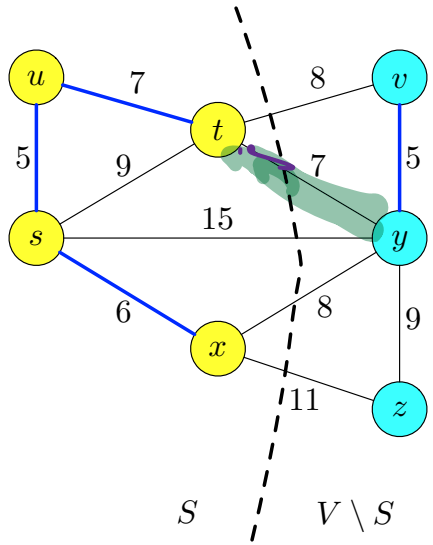
- A cut of an undirected graph is a partition $(S, V \setminus S)$ such that $S, V \setminus S \neq \emptyset$.
- An edge $(u, v) \in E(G)$ crosses the cut if $u \in S$ and $v \in V \setminus S$.
- A cut $(S, V \setminus S)$ of G respects F if no edge in F crosses $(S, V \setminus S)$.



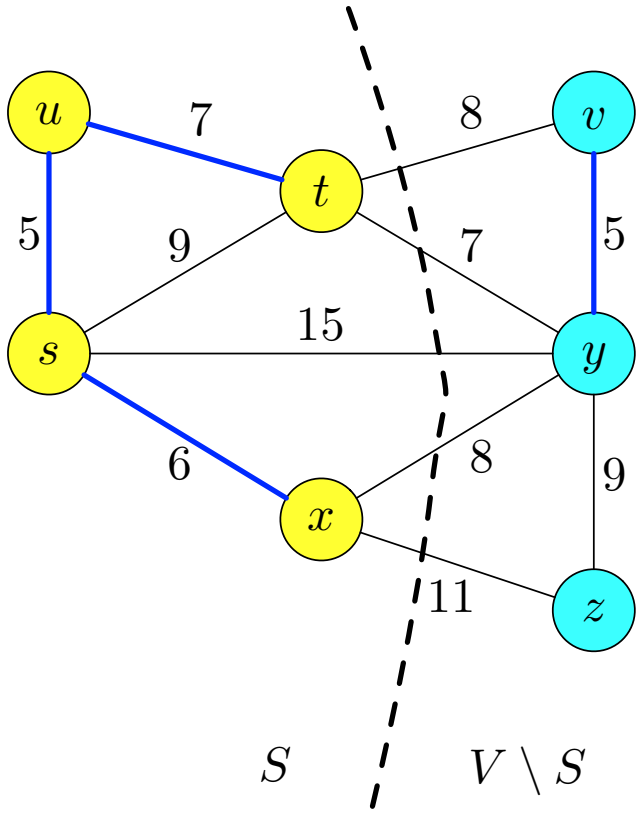


Proof (sketch) revisit

Lemma: Let $G = (V, E)$ be a weighted (nonnegative) undirected graph and $F \subseteq E$ is included in a minimum spanning tree T. Suppose that $(S, V \setminus S)$ be any cut that respects F and let $(u, v) \in E(G)$ with minimum weight that crosses $(S, V \setminus S)$. Then, $uv \in T$.



Lemma: Let $G = (V, E)$ be a weighted (nonnegative) undirected graph and $F \subseteq E$ is included in a minimum spanning tree T. Suppose that $(S, V \setminus S)$ be any cut that respects F and let $(u, v) \in E(G)$ with minimum weight that crosses $(S, V \setminus S)$. Then, $(u, v) \in T$.



- Take a minimum spanning tree T with $F \subseteq T$ and a cut $(S, V \setminus S)$ respecting F.
- Then (u, v) is safe to be added into T.
- (Proof by Contradiction): Suppose that (u, v) with smallest weight crossing $(S, V \setminus S)$ but $(u, v) \notin T$.
- Idea: construct spanning tree T' such that $\mathrm{w}(T') \leq \mathrm{w}(T)$ and $F \cup \{(u, v)\} \subseteq T'$.

Kruskal's algorithm (revisit)

$O(m \log m) = O(m \log n)$

Sort the edges in the nondecreasing order of weights;

Let $w(e_1) \leq \ldots \leq w(e_m)$ *be the nondecreasing order of weights*;

Initialize $F \leftarrow \emptyset; i \leftarrow 1$;

while *F does not form a spanning tree* **do**

  if *addition of* e_i *does not create cycle* **then**

    $F \leftarrow F \cup \{e_i\}$;

  end

  $i \leftarrow i + 1$;

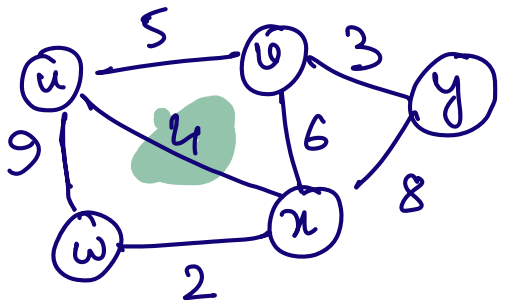
end

Return (V, F);

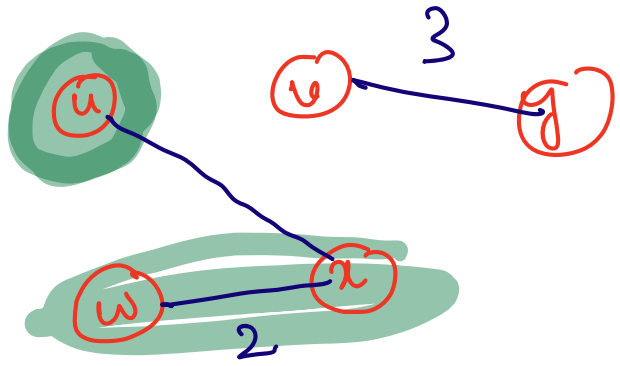
- Appropriate preprocessing and use of Union-Find Data Structure can improve the running time to $\mathcal{O}(m \log n)$-time.

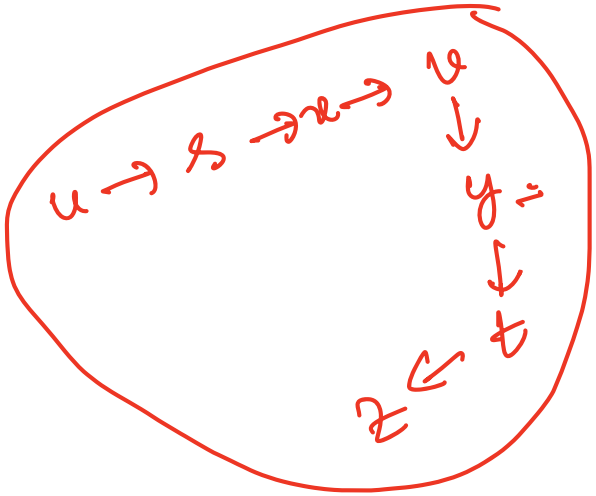
Kruskal's algorithm (revisit)

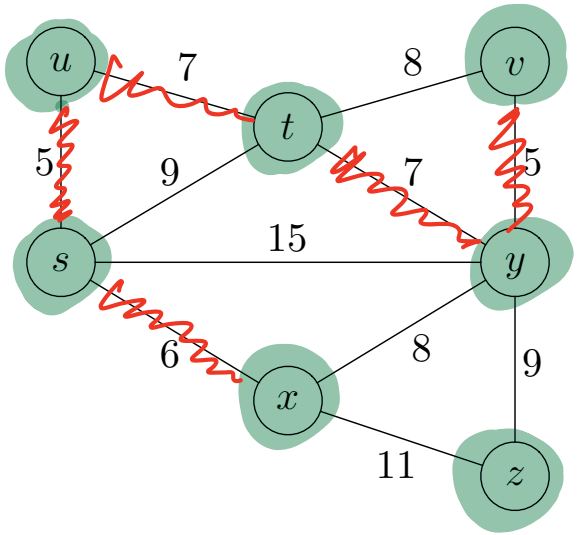
$F \leftarrow \emptyset$;
for *each vertex* $v \in V(G)$ **do**
    MakeSet(v);
end

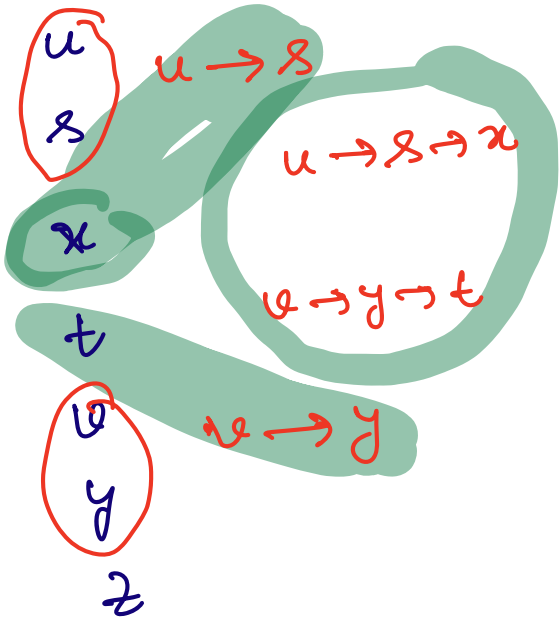


Sort the edges in the nondecreasing order of weights; $O(m \log n)$
Let $w(e_1) \leq \ldots \leq w(e_m)$ *be the nondecreasing order of weights*;
for $i = 1, \ldots, m$ **do**
    $e_i = (u, v)$;
    if FindSet(u) $\neq$ FindSet(v) **then**
        $F \leftarrow F \cup \{e_i\}$;
        Union(u, v);
    end
end
Return (V, F);









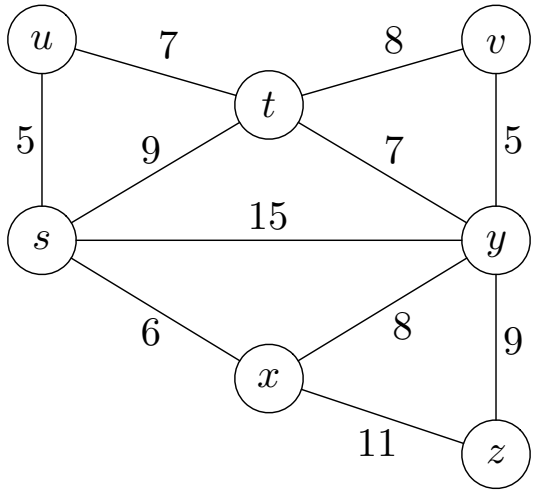
- MakeSet(x) operation creates a new set whose only member is x.
- FindSet(x) operation returns a pointer to the representative of the unique set A such that $x \in A$.
- Union(x, y) operation computes $A \cup B$ such that $x \in A$ and $y \in B$.

How many times pointer of a vertex u in a linked list gets updated?

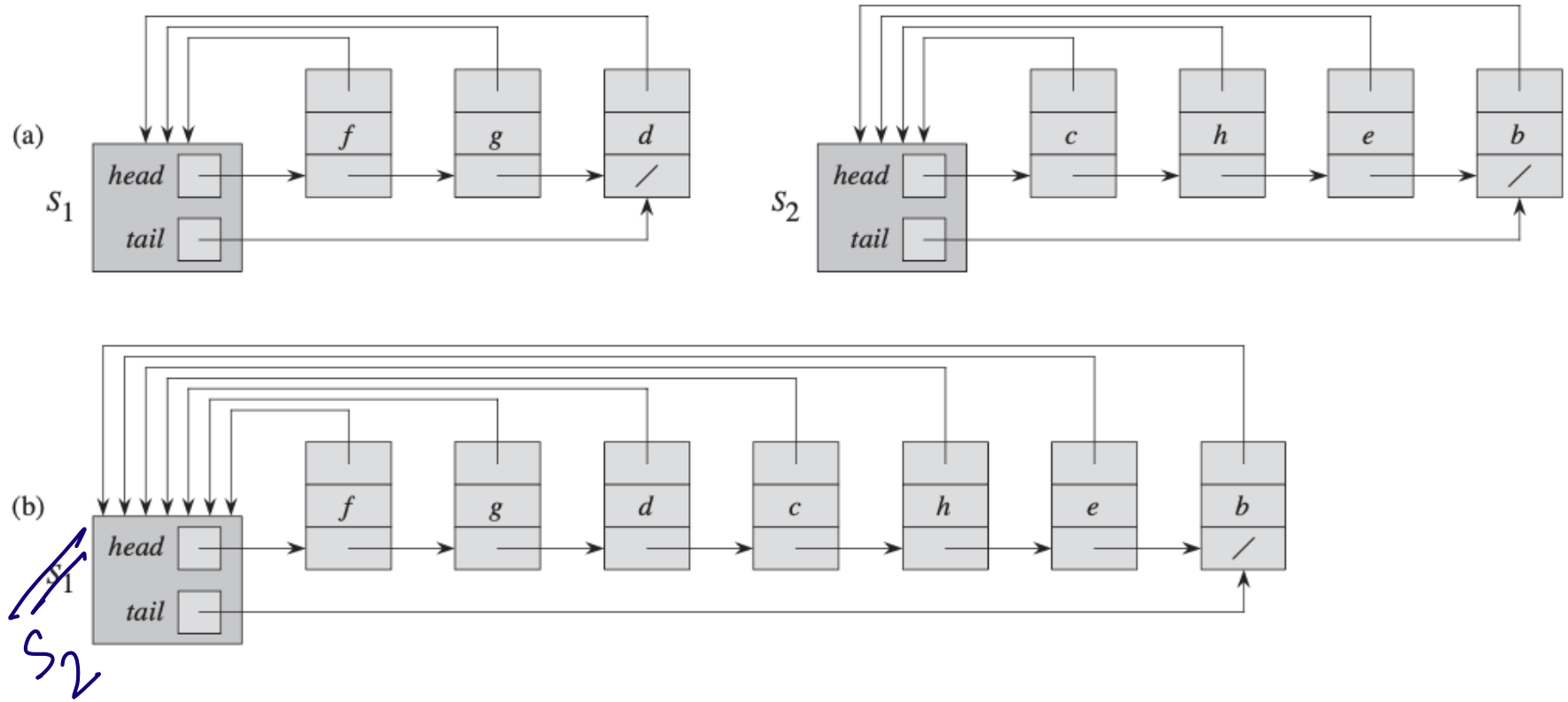
Initially u is in a linked list of size 1.

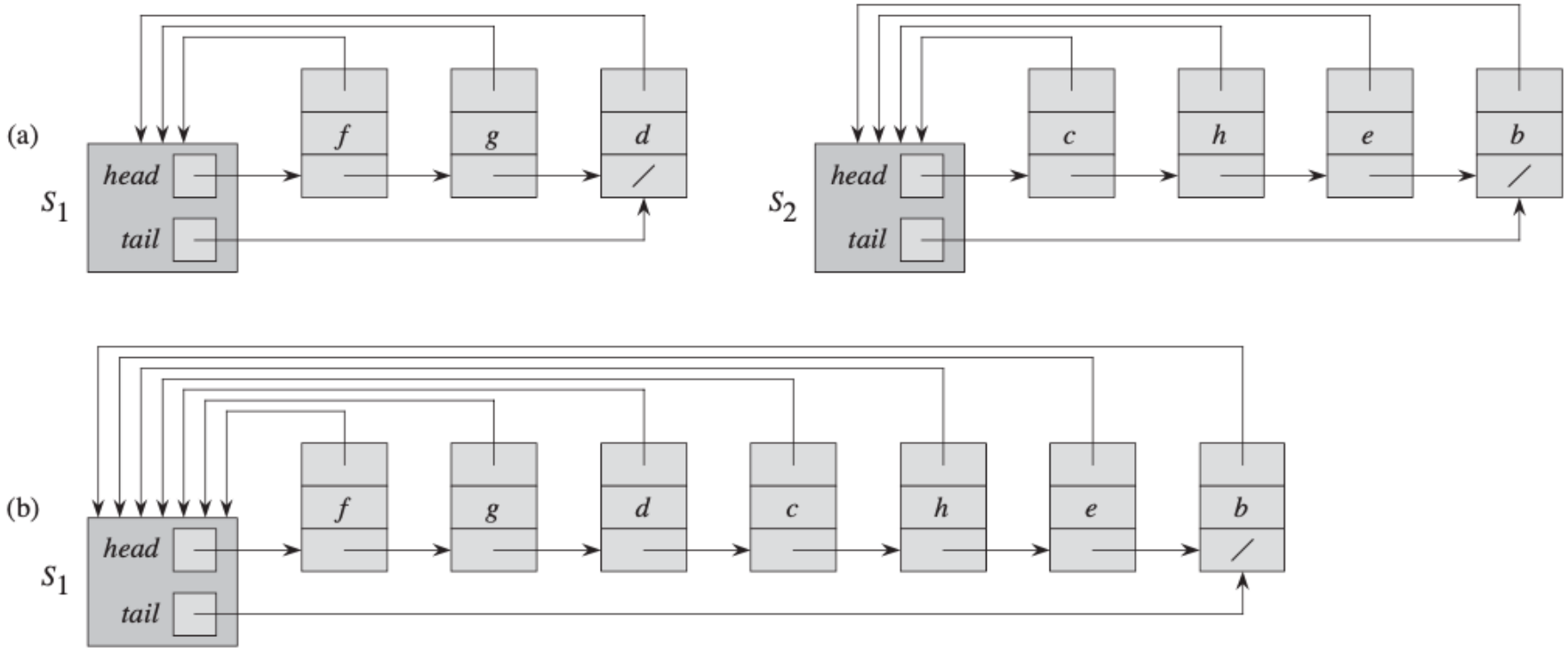
Once updated, u is in a linked list with at least

2 elements.



Linked-List Representation




(a)
S_1
head
tail
f
g
d
S_2
c
h
e
b
(b)

Clever Implementation

- Suppose that in each linked-list, we also maintain the length of the linked-list.
- We always append the shorter list onto the longer breaking ties arbitrarily.
- Analysis by the number of times the pointer for a particular object was updated.
- Consider an arbitrary vertex u. The first time, the pointer to a vertex u changes (i.e. Find-Set(u) changes), then it must be that the next time u belongs to a set of 2 elements.

UNION(x, y) step

Hence, pointer to u can get updated $O(\log n)$ times.

Kruskal's algorithm (revisit)

$F \leftarrow \emptyset$;
for *each vertex* $v \in V(G)$ **do**
    MakeSet(v);
end
Sort the edges in the nondecreasing order of weights;
Let $\mathrm{w}(e_1) \leq \ldots \leq \mathrm{w}(e_m)$ *be the nondecreasing order of weights*;
for $i = 1, \ldots, m$ **do**
    $e_i = (u, v)$;
    if FindSet(u) $\neq$ FindSet(v) **then**
        $F \leftarrow F \cup \{e_i\}$;
        Union(u, v);
    end
end
Return (V, F);

$O(m \log n)$ - time.

- Introduction to Algorithms by CLRS.
- Algorithm Design by Kleinberg and Tardos.